

14. Задание: дедупликатор файлов в S3-хранилище

Назначение

Нужно разработать **службу-дедупликатор** для объектного хранилища S3. Служба находит файлы-дубли по содержимому и устраняет лишние копии так, чтобы одинаковые данные хранились в одном экземпляре, а логические файлы оставались **независимыми**.

Главное требование: устранение дубля не должно приводить к потере данных. **Удаление или изменение одного файла не должно влиять на другие файлы, которые были его дублями.**

Основной результат

На выходе должен получиться проект s3-dedup — служба на **Go 1.20**, которая:

1. сканирует заданные бакеты и префиксы S3;
2. вычисляет хеши объектов и находит дубли по содержимому;
3. устраняет дубли так, чтобы общие данные хранились один раз;
4. хранит состояние (хеши, ссылки) в локальном кеше **SQLite** или **LevelDB**;
5. не нарушает консистентность данных при дедупликации;
6. собирается под **Windows** и **Linux (Debian/Fedora)** и запускается **как служба**;
7. конфигурируется через **YAML-файл**;
8. ведёт лог и формирует отчёт о результатах.

Как устроена дедупликация

Чтобы удаление одного файла не ломало его бывшие дубли, реальные данные хранятся отдельно от исходных ключей:

Сущность	Назначение
Logical object	Исходный ключ в S3, видимый потребителю
Blob	Уникальное содержимое, хранится один раз под ключом из хеша (blobs/<hash>)
Refcount	Число объектов, ссылающихся на этот blob

Правило консистентности — **подсчёт ссылок**:

- при дедупликации дубль заменяется ссылкой на общий blob, счётчик ссылок растёт;
- при удалении файла счётчик уменьшается;
- **blob удаляется только когда счётчик ссылок стал равен нулю**;
- порядок операций безопасный: сначала гарантированно записан blob, потом меняется или удаляется исходный объект.

Формат входных данных

Вход — объекты S3, получаемые через ListObjectsV2 по бакетам/префиксам из конфигурации. Для каждого объекта известны key, size, etag, last_modified, а содержимое читается потоково через GetObject. Хеш содержимого служба считает сама.

Формат конфигурации (YAML)

Доступы к S3 берутся из переменных окружения, а если их нет — из конфигурации. В примере доступы в конфиге опциональны и нужны в основном для локальной отладки.

```
s3:
  endpoint: "https://s3.example.local:9000"
  region: "us-east-1"
  # : S3_ACCESS_KEY / S3_SECRET_KEY.
  #
  access_key: ""
  secret_key: ""
  use_path_style: true
  buckets:
    - name: "intercepted"
      prefix: "2026/"

dedup:
  hash_algo: "sha256"
  min_size_bytes: 4096      #
  blob_prefix: "blobs/"
  mode: "report_only"      # report_only | pointer
  delete_originals: false  # :

cache:
  backend: "sqlite"        # sqlite | leveldb
  path: "/var/lib/s3-dedup/state.db"

schedule:
  scan_interval: "1h"      #
  workers: 8               #

logging:
  level: "info"
  file: "/var/log/s3-dedup/service.log"
```

Поля конфигурации:

Поле	Тип	Обязательное	Назначение
s3.endpoint	string	да	Адрес S3-совместимого хранилища
s3.access_key / s3.secret_key	string	нет	Доступы; приоритет у переменных окружения
s3.buckets[]	array	да	Бакеты и префиксы для обработки
dedup.hash_algo	string	да	Алгоритм хеширования содержимого
dedup.min_size_bytes	integer	нет	Нижний порог размера для дедупликации
dedup.mode	string	да	report_only — только отчёт; pointer — устранять дубли
dedup.delete_originals	bool	да	Разрешение удалять исходные объекты
cache.backend	string	да	Бэкенд кеша состояния: sqlite или leveldb
schedule.scan_interval	string	да	Интервал повторного сканирования

Переменные окружения:

Переменная	Назначение
S3_ACCESS_KEY	Ключ доступа (приоритетнее конфига)
S3_SECRET_KEY	Секретный ключ (приоритетнее конфига)

Выходной формат результата

Отчёт о проходе в JSON:

```
{
  "scan_started": "2026-06-16T10:00:00Z",
  "scan_finished": "2026-06-16T10:07:31Z",
  "objects_scanned": 1250000,
  "unique_blobs": 410000,
  "duplicates_found": 840000,
  "bytes_reclaimable": 560000000000,
  "bytes_reclaimed": 0,
  "objects_relinked": 0,
  "errors": 12,
  "mode": "report_only"
}
```

Поле	Назначение
objects_scanned	Сколько объектов обработано
unique_blobs	Число уникальных по содержимому файлов
duplicates_found	Сколько объектов оказались дублями
bytes_reclaimable	Сколько места можно освободить (режим отчёта)
bytes_reclaimed	Сколько фактически освобождено (режим pointer)
objects_relinked	Сколько дублей заменено ссылками
errors	Число ошибок обработки
mode	Режим прохода

Служба и CLI

Обязательные команды:

```
s3-dedup run --config config.yml # ( )
s3-dedup scan-once --config config.yml #
s3-dedup report --config config.yml --out report.json
```

Управление службой:

```
s3-dedup install --config config.yml # (Windows Service / systemd)
s3-dedup uninstall
```

Служба должна корректно завершаться по сигналу остановки: доделать текущую безопасную операцию, сохранить состояние и не оставить хранилище в промежуточном виде.

Требования к консистентности

1. Повторный запуск на уже обработанном хранилище ничего не портит и не создаёт лишних blob'ов (идемпотентность).
2. Сначала гарантированно записан blob, и только потом меняется или удаляется исходный объект.
3. Blob удаляется только при счётчике ссылок, равном нулю.
4. После перезапуска службы состояние восстанавливается из кеша без потери данных.
5. Если исходный файл изменился (новый хеш), это новый контент: старый blob теряет одну ссылку.
6. Ошибка при обработке одного объекта не должна останавливать весь проход.

Требования к ресурсам и вводу/выводу

1. Содержимое хешируется **потоково**, без загрузки всего файла в память.
2. Список объектов обрабатывается постранично, без загрузки всего листинга в память.
3. Память службы при обходе 1 000 000 объектов — не более 512 МБ.

4. Хеширование распараллеливается на workers воркеров.
5. Состояние сохраняется в кеш так, чтобы переживать перезапуск.

Алгоритмические требования

Нужно реализовать:

1. потоковое хеширование объекта (`io.Reader hash`);
2. хранение «хеш blob счётчик ссылок» в кеше (SQLite или LevelDB);
3. поиск дублей по совпадению хеша;
4. устранение дубля заменой на ссылку с увеличением счётчика;
5. безопасное удаление blob при нулевом счётчике;
6. восстановление состояния после перезапуска.

Критерии приёмки

Минимум:

1. чтение конфигурации из YAML и доступов из переменных окружения;
2. сканирование одного бакета/префикса;
3. потоковое хеширование и поиск дублей;
4. режим `report_only` с JSON-отчётом;
5. хранение хешей и счётчиков в кеше (SQLite **или** LevelDB);
6. сборка под Windows и Linux;
7. unit-тесты хеширования и логики счётчика ссылок.

Хорошо:

1. режим `pointer` с безопасной заменой дублей на ссылки;
2. удаление blob только при нулевом счётчике ссылок;
3. запуск как служба (`systemd` и/или Windows Service);
4. корректное завершение и восстановление состояния после перезапуска.

Отлично:

1. идемпотентность, подтверждённая тестом «повторный проход ничего не меняет»;
2. тест на устойчивость: имитация падения в середине обработки без потери данных;
3. поддержка обоих бэкендов кеша (SQLite и LevelDB) с выбором через конфиг.

Достаточно выбрать 1–2 пункта уровня «Отлично» и сделать их качественно, с тестами.

Общие требования к сдаче

Решение оформляется как самостоятельный проект на Go (модуль, go 1.20).

Обязательная структура проекта:

```
/cmd/s3-dedup      CLI
/internal/...      (s3, hashing, cache, dedup, service)
/configs           config.yml
/testdata
/docs
README.md
Makefile           test, build, demo
```

Обязательные команды:

```
make test
make build      # windows linux
make demo
```

В README.md должны быть описаны:

1. назначение проекта;
2. сборка под Windows и Linux и установка как службы;

3. формат конфигурации YAML и переменные окружения;
4. формат отчёта;
5. примеры команд;
6. краткое описание схемы дедупликации и гарантий консистентности;
7. известные ограничения;
8. результаты контрольного запуска.

Контрольная сдача и оценивание

В репозитории должен быть фиксированный набор `/testdata/control`: небольшой набор объектов (часть — заведомые дубли), конфигурация и ожидаемый отчёт. Для запуска допускается локальный S3-эмулятор (например, MinIO); способ запуска должен быть описан. Команда `make demo` запускает решение на этом наборе и сохраняет результат в предсказуемые файлы.

Оценка решения:

Критерий	Вес
Корректность дедупликации и консистентность	40%
Тесты, включая повторные проходы и граничные случаи	20%
Сборка и работа как службы (Windows/Linux)	15%
Качество конфигурации, README и воспроизводимость	15%
Простота архитектуры и читаемость кода	10%

В `/docs/reshenie.md` нужно на 1–2 страницы описать выбранную схему дедупликации, как обеспечивается консистентность (порядок операций, счётчик ссылок, восстановление после сбоя), выбор кеша, известные ограничения и идеи для улучшения.

Требования к исходному коду и истории коммитов

1. Код читаемый: понятные имена, разделение по пакетам, без «мёртвого» кода.
2. Ошибки возвращаются явно и с контекстом (какой объект/операция).
3. Никаких `panic` на штатных и некорректных входных данных.
4. Осмысленные атомарные коммиты, описывающие изменения логики; без сообщений вида `fix`, `wip`, `final2`.
5. `gofmt/go vet` без замечаний; проект собирается командой `make build`.

Справочные материалы

- [Дедупликация данных](#) — устранение избыточных копий данных.
- [Content-addressable storage](#) — хранение по хешу содержимого.
- [Подсчёт ссылок](#) — управление временем жизни общих данных.
- [Amazon S3 API \(ListObjectsV2, GetObject\)](#) — операции над объектным хранилищем.